

Documento dei Requisiti

Progetto: Gestione Prestiti di una Biblioteca Personale

Corso:	Informatica / Programmazione JavaScript
Scadenza:	29 Aprile 2026
Versione:	2.0
Data documento:	28 Aprile 2026

1. Introduzione

Il presente documento descrive i requisiti funzionali e non funzionali dell'applicazione **Gestione Prestiti di una Biblioteca Personale**, sviluppata in JavaScript. L'obiettivo del sistema e di consentire all'utente di gestire i libri della propria collezione privata, monitorarne la disponibilit  e tracciare i movimenti di prestito e restituzione.

2. Requisiti Funzionali

I requisiti funzionali descrivono le operazioni che il sistema deve essere in grado di eseguire. Sono classificati in base alla versione di implementazione.

2.1 Versione Base

ID	Requisito	Descrizione	Priorita
RF01	Registrazione libro	Il sistema deve permettere l'aggiunta di un nuovo libro con codice identificativo univoco, titolo e autore.	Alta
RF02	Prestito libro	Il sistema deve associare un libro a un lettore registrando nome del lettore e data del prestito.	Alta
RF03	Restituzione libro	Il sistema deve aggiornare lo stato di un libro da 'in prestito' a 'disponibile' e registrare la data di restituzione.	Alta

2.2 Versione Avanzata

ID	Requisito	Descrizione	Priorita
RF04	Filtra disponibili	Il sistema deve permettere di visualizzare solo i libri con stato 'disponibile'.	Media
RF05	Filtra in prestito	Il sistema deve permettere di visualizzare solo i libri attualmente in prestito.	Media
RF06	Gestione ritardi	Il sistema deve evidenziare i prestiti che superano un numero configurabile di giorni (default 30).	Alta
RF07	Storico operazioni	Il sistema deve conservare la cronologia completa di tutte le operazioni: aggiunte, prestiti e restituzioni.	Alta

3. Requisiti Non Funzionali

I requisiti non funzionali definiscono le qualita e i vincoli del sistema, indipendentemente dalle funzionalita specifiche.

ID	Categoria	Descrizione
----	-----------	-------------

RNF01	Modularita	Il codice deve essere organizzato in moduli separati (Libro.js, Prestito.js, Storico.js, Biblioteca.js).
RNF02	Manutenibilita	Ogni classe e metodo deve essere documentato con commenti JSDoc completi (descrizione, @param, @return).
RNF03	Affidabilita	Il sistema deve gestire correttamente i casi di errore (libro non trovato, prestito su libro gia in prestito).
RNF04	Leggibilita	Il codice deve seguire convenzioni di naming chiare (camelCase per metodi/variabili, PascalCase per classi).
RNF05	Portabilita	Il sistema deve funzionare su qualsiasi piattaforma con Node.js installato, senza dipendenze esterne.
RNF06	Versionamento	Il progetto deve essere gestito con Git, con commit regolari e descrittivi che documentino l'avanzamento.
RNF07	Scalabilita	L'architettura a classi deve permettere l'aggiunta futura di nuove funzionalita (es. interfaccia grafica).

4. Architettura del Sistema

Il sistema adotta un'architettura **Object-Oriented** basata su quattro classi principali:

Classe / Modulo	Responsabilita
Libro.js	Rappresenta un singolo libro con attributi e stato (disponibile / in prestito).
Prestito.js	Modella un prestito attivo o storico, con calcolo automatico dei giorni e rilevamento ritardi.
Storico.js	Registra e recupera la cronologia delle operazioni effettuate sulla biblioteca.
Biblioteca.js	Classe principale che orchestra tutte le operazioni, gestendo il catalogo e delegando allo Storico.
main.js	Punto di ingresso: istanzia la Biblioteca e dimostra tutte le funzionalita disponibili.

5. Materiale Consegnato

#	File / Artefatto	Descrizione
1	UML_Biblioteca.png	Diagramma UML delle classi del sistema.
2	Requisiti_Biblioteca.pdf	Questo documento: requisiti funzionali e non funzionali.
3	Libro.js	Classe Libro con JSDoc.
4	Prestito.js	Classe Prestito con JSDoc.
5	Storico.js	Classe Storico con JSDoc.
6	Biblioteca.js	Classe principale Biblioteca con JSDoc.
7	main.js	Punto di ingresso e demo dell'applicazione.
8	link.txt	URL del repository GitHub pubblico.